```
print "Doing something explode-y" . PHP_EOL;

try {
    throw new Exception('Boom!');
} catch (Exception $e) {
    print "Exception caught!" . PHP_EOL;
} finally {
    print "Sweeping up the mess..." . PHP_EOL;
}

print "... and carrying on.";

Output:
Doing something explode-y
Exception caught!
Sweeping up the mess...
... and carrying on.
```

One limitation of `finally` is that it cannot catch an exception for you, and although it will always execute if something throws one in your `try` block, an uncaught exception is still going to kill the script.

Exception Handling

Uncaught exceptions are handled by the (appropriately named) exception handler, which is some "magic" code that sits outside of your own program. Just as you can define your own custom error handler (albeit for limited error levels), so too can you define one for uncaught exceptions. As we've seen earlier in this section, an uncaught `Error` exception resulted in an `E_ERROR` being emitted; this would be the case for any throwable object of class `Exception` or `Error` – or a class that implements the `Throwable` interface. Interfaces are a definition of how a class behaves in terms of its public-facing properties and methods. When a class is declared as implementing an interface – `class Error implements Throwable`, for example – it must provide implementations of whatever `interface Throwable` defines. We'll go no further than that; just accept that PHP keeps track of all classes, their inheritance hierarchy, and the interfaces of both the class itself and the inheritance hierarchy, and interfaces can be used in type declarations.